Doomscroll

Software Requirements Specification — Lab 5 (Updated)

Version 2.0 -- supersedes Lab 1's Requirements Specification (v1.0) and the requirements referenced by Lab 2's Use Case Model (v1.1)
Prepared by: Project Team -- SC2006
Date: 6 November 2026

Revision History

Version	Date	Author	Reason for Changes
1.0	6 September 2026	Project Team	Initial Requirements Specification (Lab 1).
1.1	20 September 2026	Project Team	Use Case Model reviewed for consistency with the Lab 2 conceptual model and dialog map; no requirement changes.
2.0	6 November 2026	Project Team	Lab 5 update: requirements and the Use Case Model rewritten to match the current codebase. Several Lab 1/1.1 requirements described behaviour that was never implemented and have been corrected or removed; see “Changes Since Lab 2” below.

Changes Since Lab 2

This revision was produced by reading the current implementation end-to-end (the launcher in app.py, the gaze/calibration pipeline, the content-aware scroll trigger, the platform scroll/window backends, and the recording/training pipeline) and correcting the requirements and Use Case Model wherever they described behaviour the codebase does not actually have. The changes, in order of impact:

- The “Important Content” / “Unimportant Content” binary classification used throughout Labs 1-2 has been replaced with the real decision the system makes: a 3-tier Scroll Trigger (bottom-of-window / reached-end-of-visible-text / nothing-unseen-ahead), implemented in src/doomscroll/fixation/scroll_trigger.py. There is no separate step that labels regions “important”; OCR only ever reports text blocks and their positions.
- Content-awareness (tiers 2 and 3 of the Scroll Trigger) is macOS-only, via Apple Vision OCR (src/doomscroll/content/macos_backend.py). On Windows, or any machine without that backend available, the system falls back to tier 1 (bottom-of-window) only -- this was not disclosed anywhere in Lab 1/2, which claimed full Windows 10+ / macOS 12+ parity (old Requirement 4.6.1).
- Direction toggles, sensitivity, and speed settings (old Requirements 3.5.1-3.5.3, old Use Case 5.1) do not exist. The live pipeline always scrolls downward only, by a fixed step size (SCROLL_STEP_LINES = 6 lines); the Settings page offers exactly two controls: Appearance (Light/Dark) and Clear all calibrations.
- The keyboard-shortcut / on-screen pause-resume control (old Requirement 3.5.4, old Use Case 5.2) does not exist. The closest real behaviour is an automatic Attention Gate (src/doomscroll/fixation/attention_gate.py) that pauses scrolling, with no action from the User, whenever a coarse head-angle check detects they have turned or tilted away from the screen, and resumes automatically once they turn back. Ending a running session is done by closing its window (pressing 'q'), which is not a pause -- it returns to the Main Page.
- The automatic “sustained low Confidence Score → recalibration prompt” behaviour (old Requirement 3.1.5, old Use Case 1.3) does not exist. No per-frame Confidence Score is computed or exposed outside the Math Model's own internal blending step (which never leaves GazeCalibrator.predict()). Recalibration is always a deliberate action the User takes from the Main Page; the only automatic prompt is the Calibration Checkpoint dialog that follows every calibration run and asks whether to proceed or redo it, based on the mean-pixel validation error the run itself just printed.
- “Manual Scroll suppresses Autoscroll” (old Requirements 3.4.5-3.4.6, old Use Case 4.2) does not exist -- there is no manual-scroll detection anywhere in the codebase. This is recorded as a known gap (Section 6) rather than silently dropped.
- The system actually ships two interchangeable Gaze Models with two different Calibration procedures -- a Math Model (GazeCalibrator, fit fresh each session) and a CNN Model (TrainedGazeModel, trained offline and refined by a small per-user correction) -- which Labs 1-2 did not distinguish at all. A new functional requirement group (3.7) covers the CNN Model's own Recording Session and retraining pipeline, which Labs 1-2 never described.

- A privacy requirement has been added (3.6.5 / 4.3.2) reflecting a deliberate design decision: the system never collects, displays, or stores the User's real name anywhere, including in the Training Dataset -- only a random Participant ID generated once per machine.

1. Project Summary

Doomscroll is a desktop application that scrolls the content on the user's screen automatically, by tracking the user's head movement and eye gaze through the device's built-in or USB webcam. The system continuously estimates the point on the screen the user is currently looking at (the Gaze Point) using one of two interchangeable Gaze Models, determines where that point falls relative to the currently focused window and (on macOS) the text content visible within it, and performs Autoscroll -- scrolling that window downward -- once a 3-tier Scroll Trigger decides the user has read down to the bottom of what is currently visible.

The project is distributed as a cloned Git repository rather than a packaged installer: a user who clones it gets a bundled face-landmark model and (once committed, see Section 6) a baseline anonymized training dataset and pretrained CNN, and runs app.py as the entry point.

2. Target Users

The primary user is a sighted adult using a desktop or laptop computer, running Windows or macOS, with a functioning webcam, who reads long-form on-screen content such as articles, documentation, code, or PDFs, and who wants to reduce repetitive manual scrolling while reading.

Users are assumed to be seated at a roughly fixed distance and angle from the screen and webcam, in normal indoor lighting, consistent with typical desk use.

A user on Windows (or on macOS without the optional Vision OCR dependency available) gets the same Gaze-tracking and Autoscroll behaviour, but only tier-1 (bottom-of-window) scrolling -- content-aware scrolling (tiers 2-3) is macOS-only, since it depends on Apple's Vision framework. Mobile and touchscreen devices, users without a webcam, and users with visual or motor impairments that prevent reliable webcam-based gaze estimation are out of scope for this phase of the project. A dedicated accessibility evaluation is recommended for a later phase.

3. Functional Requirements

3.1 Camera Access

3.1.1 The system shall access the device's webcam through the operating system's own camera framework (AVFoundation on macOS, DirectShow on Windows) whenever a tracking-dependent step begins (Calibration, Recording, or Autoscroll). The system does not display its own explanation of why camera access is needed; the User grants or denies access entirely through the operating system's own permission prompt, the first time a step tries to open the camera.

3.1.2 If the webcam cannot be opened or a frame cannot be read -- including because camera permission was denied -- the system shall stop the step currently running, report the failure in the Log, and return the User to the Main Page, rather than starting or continuing Autoscroll.

3.2 Head and Gaze Tracking

3.2.1 The system shall continuously detect face landmarks from each webcam frame using the bundled MediaPipe face-landmark model (models/face_landmarker.task).

3.2.2 The system shall continuously estimate the User's Head Pose (yaw, pitch, roll, and head translation) from the detected landmarks.

3.2.3 The system shall exclude any frame in which the eyes are assessed as closed (an eye-aspect-ratio below a configurable floor) from Gaze Point computation, so that blinking does not feed a biased reading into tracking.

3.2.4 The system shall compute a Gaze Point for each usable frame using exactly one of two interchangeable Gaze Models, selected once per run:

3.2.4.1 the Math Model (GazeCalibrator) -- a per-session ridge-regression fit over Head Pose and an eye-region feature vector; or

3.2.4.2 the CNN Model (TrainedGazeModel) -- a convolutional network trained offline on the pooled Training Dataset, refined by a small per-user Affine Correction.

3.2.5 The system shall smooth each computed Gaze Point with a Kalman filter followed by an exponential moving average before using it for any downstream decision.

3.2.6 The system shall clamp a raw Gaze Point prediction to within 50% of the screen's width/height beyond each screen edge before smoothing, bounding the effect of an out-of-range model extrapolation.

3.3 Calibration

3.3.1 Quick Start (Math Model): the system shall let the User choose between two calibration protocols -- a single continuously-moving dot followed on-screen for about 40 seconds (Trajectory, the default), or a 16-point on-screen grid fixated one point at a time (Dots) -- and shall fit a new Calibration Profile from the samples captured, plus a short head-pose-variation pass (lean/tilt) for either protocol.

3.3.2 Quick Start: the system shall let the User optionally pool the current session with their own most recent stored calibration sessions of the same screen size and protocol, and shall keep whichever fit -- session-only or pooled -- validates with the lower mean pixel error, reporting both.

3.3.3 Full Start / Calibrate only (CNN Model): the system shall fit a per-user Affine Correction on top of the existing trained CNN using a fixed 9-point grid (~15 seconds), rather than refitting the CNN itself.

3.3.4 After every Calibration or Quick Calibrate run, in every flow, the system shall ask the User whether to proceed to Autoscroll with that result, redo the calibration, or cancel, before Autoscroll can start.

3.3.5 The system shall let the User discard, in one action, all stored Math Model Calibration Profiles, the CNN Model's per-user correction, and all pooled calibration sessions.

3.4 Content-Aware Scroll Trigger

3.4.1 The system shall determine which on-screen window currently has operating-system focus, on every frame, and shall use that window's bounds -- not the whole screen -- as the Autoscroll viewport.

3.4.2 Where supported (macOS only, via Apple Vision OCR), the system shall periodically (about every 0.4 seconds) scan the focused window for Content Blocks: regions of recognized text together with their bounding boxes.

3.4.3 The system shall decide, on every frame with a usable Gaze Point, whether to scroll using the first of the following that applies:

3.4.3.1 Tier 1 -- the Gaze Point has dwelt in the bottom 15% of the focused window.

3.4.3.2 Tier 2 -- OCR shows the Gaze Point has reached and moved past the end of the lowest Content Block currently visible.

3.4.3.3 Tier 3 -- OCR shows nothing unseen remains between the Gaze Point and the bottom of the window, after a longer, more conservative dwell than tier 1.

3.4.4 Where OCR is unavailable -- any non-macOS platform, or a macOS installation without the Vision dependency -- the system shall fall back to tier 1 only, rather than disabling Autoscroll entirely.

3.4.5 The dwell/“seen” tracking behind tiers 2-3 shall tolerate the Calibration system's typical gaze error: a Gaze Point within a configurable pixel distance of a Content Block shall count as having reached it, and a brief grace period shall be allowed before a momentary divergence resets dwell progress.

3.5 Scrolling and Interruption

3.5.1 When the Scroll Trigger fires, the system shall scroll the focused window downward by a fixed step, using the operating system's native scroll-wheel event mechanism (Quartz on macOS, the Windows mouse-event API on Windows), briefly moving the cursor to the window's center so the event is delivered to the right window.

3.5.2 The system shall pause all scrolling, independently of the Scroll Trigger, whenever the Attention Gate detects the User's head has turned or tilted beyond a configured angle, and shall resume automatically, with no action from the User, once their head returns within range.

3.5.3 The User shall be able to end a running Autoscroll session at any time by closing its window, returning to the Main Page.

3.6 Application Control and Configuration

3.6.1 The system shall let the User choose, from the Main Page, between Quick Start (Math Model), Record, train & calibrate (CNN Model, Full Start), and Calibrate only (CNN Model).

3.6.2 The system shall let the User switch the launcher's own appearance between Light and Dark; this choice applies for the current launch only and is re-derived from the operating system's current appearance setting the next time the application starts.

3.6.3 The system shall show the combined console output of every step of the current flow in a single scrollable Log on the Settings page, updated in real time regardless of which page is currently shown.

3.6.4 The system shall disable the Main Page's action buttons while a flow is running, and re-enable them once it finishes, fails, or is cancelled.

3.6.5 The system shall never collect, display, or request the User's real name. It shall identify a User's own Recording Sessions and Calibration Profiles using a random Participant ID, generated once per machine the first time it is needed and reused automatically after that.

3.7 Recording and Model Training (Full Start)

3.7.1 The system shall let the User record a Recording Session of a duration they choose, during which the User follows a cursor that moves around the screen.

3.7.2 After a Recording Session, the system shall extract per-frame face crops and numeric feature vectors from it into the permanent, anonymized Training Dataset (datasets/crops/, datasets/features/), then delete the raw recording.

3.7.3 The system shall re-estimate the CNN Model's cursor Lag Estimate from the full Training Dataset after every new Recording Session, on a best-effort basis: if this step fails, the rest of the flow continues using whichever Lag Estimate was already stored.

3.7.4 The system shall retrain the CNN Model from the full Training Dataset, using the current Lag Estimate, after every new Recording Session.

4. Non-Functional Requirements

4.1 Performance

4.1.1 The system is designed to compute a new Gaze Point at least 15 times per second on a reference mid-range laptop (quad-core processor, no dedicated graphics card required); this target is not currently enforced or measured by an automated benchmark in the codebase.

4.2 Accuracy

4.2.1 After Calibration, the Gaze Point is targeted to be accurate to within approximately 10% of screen width and height of the User's true gaze location, under normal indoor lighting and a seated desk posture. Each Calibration run reports its own mean pixel validation error so the User can judge this directly before proceeding.

4.3 Privacy and Data Protection

4.3.1 The system shall process all webcam images locally on the User's device and shall not transmit them, or any value derived from them, to an external server; the codebase makes no network calls that do so.

4.3.2 The system shall not retain raw webcam video beyond a Recording Session's own temporary staging area; the permanent Training Dataset holds only derived face crops and numeric feature vectors, keyed by the random Participant ID (Requirement 3.6.5), never the User's name.

4.4 Usability

4.4.1 A first-time user is targeted to be able to complete the default Quick Start Calibration (Trajectory protocol, roughly 40 seconds plus a short head-pose pass and validation) and begin using Autoscroll within about 2 minutes, guided only by on-screen prompts.

4.5 Reliability

4.5.1 When the User's face cannot be detected, a usable gaze feature cannot be extracted (Math Model), or the User is mid-blink, the system shall withhold a Gaze Point for that frame and reset the Scroll Trigger's dwell state, rather than issue a stale or incorrect scroll.

4.6 Compatibility

4.6.1 The system shall run on Windows 10 (or later) and macOS computers equipped with an integrated or USB webcam, requiring Python 3.11 or 3.12. Gaze tracking, both Calibration paths, and Autoscroll (tier 1) are available on both platforms; content-aware Autoscroll (tiers 2-3) is available on macOS only, since it depends on Apple's Vision framework.

4.7 Resource Usage

4.7.1 The system is targeted not to use more than 30% average processor capacity or 500 MB of memory during continuous operation on the reference hardware described in Requirement 4.1.1; this target is not currently measured or enforced by the codebase.

4.8 Scope and Portability

4.8.1 The system is not required to support mobile or touchscreen devices in this phase of the project.

4.8.2 The system is not required to support user-configurable scroll direction, sensitivity, or speed, or detection and suppression of the User's own manual scrolling, in this phase of the project; see Section 6 for these as explicit known gaps rather than silent omissions.

4.9 Robustness to Involuntary Movement

4.9.1 The system shall exclude webcam frames in which the User's eyes are detected as closed from Gaze Point computation, so that blinking does not trigger Autoscroll.

4.10 Configurability and Persistence

4.10.1 The system shall retain each User's Math Model Calibration Profile, CNN Model per-user correction, pooled calibration sessions, and Participant ID between launches, using the operating system's standard per-application data directory. The Appearance choice (Requirement 3.6.2) is the only setting that does not persist as an override.

5. Glossary

Term	Definition
Gaze Point	The screen-pixel coordinate the system currently estimates the User is looking at, produced by whichever Gaze Model is active that run.
Gaze Model	One of two interchangeable pipelines that turn a webcam frame and Head Pose into a Gaze Point: the Math Model (GazeCalibrator) or the CNN Model (TrainedGazeModel).
Calibration Profile	The fitted parameters of a Gaze Model: a Math Model profile file, or the CNN Model's per-user Affine Correction.
Head Pose	Yaw, pitch, roll, and translation estimated from face landmarks each frame.
Attention Gate	A coarse head-angle check that automatically pauses all scrolling when the User visibly turns or tilts away from the screen, and resumes it when they turn back.
Focused Window	The bounds and owning application name of the operating system's current frontmost window; used as the Autoscroll viewport instead of the whole screen.
Content Block	One OCR-recognized block of on-screen text, with its bounding box, reported by the macOS-only Vision-based content scanner.
Dwell	The system's tracking, over time, of which Content Blocks the Gaze Point has lingered on or near -- the basis for deciding the User has already read down to a point.

Scroll Trigger	The 3-tier decision (bottom-of-window / reached-end-of-content / nothing-unseen-ahead) that decides, each frame, whether to scroll.
Recording Session	A long webcam capture, following a moving on-screen cursor, used to extend the CNN Model's Training Dataset.
Training Dataset	The permanent, anonymized archive of per-frame face crops and feature vectors (datasets/crops/, datasets/features/) built from Recording Sessions.
Lag Estimate	A millisecond offset, re-computed from the Training Dataset, correcting for the delay between a captured frame and the on-screen position it reflects; fed into the CNN Model's retrain.
Participant ID	A random identifier generated once per machine and reused automatically, used to group a User's own Recording Sessions and Calibration Profiles. The system never collects or stores a User's real name.

6. Known Gaps / Out of Scope

The following were described as implemented in Lab 1/Lab 2 but are not present in the current codebase. They are recorded here, rather than silently dropped, so that traceability between requirements, use cases, and the implementation stays honest for Lab 5's demo and submission.

- Manual-scroll detection and suppression (old Requirements 3.4.5-3.4.6): there is no code anywhere in the project that detects a User's own mouse/trackpad/keyboard scroll and temporarily suppresses Autoscroll around it. A manual scroll and an Autoscroll tick could in principle fire back-to-back.
- User-configurable scroll direction, sensitivity, and speed (old Requirements 3.5.1-3.5.3): the ScrollInjector interface (src/doomscroll/scroll/base.py) supports an arbitrary dx/dy, but the live pipeline (scripts/phase2_gaze_scroll.py) always calls it with a fixed downward step; none of this is exposed as a setting.
- Content-aware scrolling on Windows: src/doomscroll/content/__init__.py raises NotImplementedError for any platform other than macOS, so a Windows user always runs in tier-1-only mode, with no Content Block information available at all.
- A persisted Appearance override: the Light/Dark choice is re-derived from the operating system's current setting on every launch rather than remembered as an explicit User preference.

7. Traceability Note

Each functional requirement above is stated as a single, independently identifiable capability and is traceable to at least one Use Case in the companion Use Case Model document (use_case_model.pdf), via that use case's “Special Requirements” field. Requirement numbers are referenced directly wherever they are covered by a use case, so that traceability from requirement to design to test can be demonstrated without ambiguity.